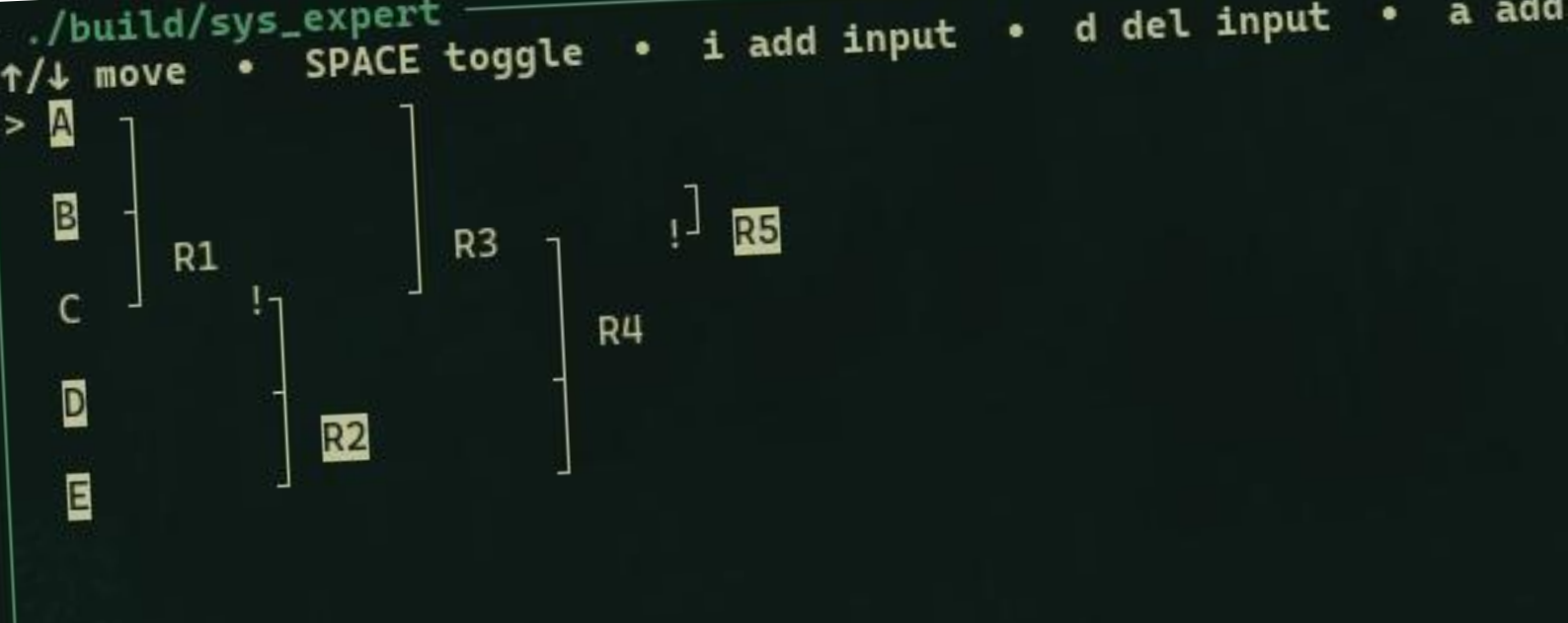




UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

LO21 : Système expert

Rapport sur Projet LO21 - A2025

GUYENOT-COSIO Amber

Tronc Commun

UV LO21 : Algorithmique et programmation :
niveau II

Responsable et enseignants :

Abderrafiaa KOUKAM

Ibrahim KAJO

LO21-A2025 : GUYENOT-COSIO

Table des matières

TABLE DES MATIERES	2
INTRODUCTION	4
Résumé fonctionnel :	4
Démarche et organisation :	4
CHOIX DE CONCEPTION ET STRUCTURES DE DONNEES.....	5
Type Abstrait « Liste »	5
Structures de données :	5
Opérations disponibles (respectant les principes du cours) :	5
Invariants :	5
Type Proposition	6
Représentation choisie :	6
Identifiant et égalité :	6
Gestion de la mémoire :	6
Type Abstrait Règle	7
Structure :	7
Opérations principales :	7
Invariants :	7
Type Abstrait BC (Base de Connaissances).....	7
Structure :	7
Opérations :	7
Choix de conception :	8
Démarche d'Implémentation.....	8
Pourquoi ces structures :	8
Complexité attendue :	8
Compromis :	8
ALGORITHMES DES SOUS-PROGRAMMES DEMANDES.....	10
Type Abstrait Règle	10
Créer une règle vide	10
Ajouter une proposition à la prémisse (en queue)	10
Créer la conclusion d'une règle	10
Tester l'appartenance d'une proposition à la prémisse (récursif)	11
Supprimer une proposition donnée de la prémisse	11
Tester si la prémisse est vide	11
Accéder à la proposition en tête de la prémisse	12
Accéder à la conclusion d'une règle	12
Vérifier si une règle possède une conclusion	12
Type Abstrait BC (Base de Connaissances).....	13
Créer une base vide.....	13
Ajouter une règle à une base (en queue)	13
Accéder à la règle en tête de la base.....	13
Supprimer une règle par le nom de sa conclusion	14

Type Abstrait Base De Faits	15
Créer une base de faits vide	15
Ajouter un fait à la base	15
Tester si un fait appartient à la base.....	15
Vérifier si la prémisse d'une règle est satisfaite	16
Moteur d'Inférence par Chaînage Avant	17
Notes	17
INTERFACE UTILISATEUR INTERACTIVE	18
Motivation et Objectifs	20
Choix Technologiques	20
Bibliothèque ncurses :	20
Fallback en mode texte simple :	20
CONCLUSION	21

Introduction

Ce rapport présente la conception, les algorithmes et la démarche d'implémentation d'un système expert élémentaire réalisé en langage C dans le cadre du module LO21. L'objectif est de modéliser une base de connaissances sous forme de règles, de manipuler une base de faits et d'exécuter un moteur d'inférence par chaînage avant afin de déduire de nouveaux faits certains. Le projet s'appuie sur des types abstraits de données étudiés en cours, en particulier une liste séquentielle utilisée pour représenter les prémisses des règles, l'ensemble des règles d'une base de connaissances et la base de faits.

Résumé fonctionnel :

- Une règle est composée d'une prémisses (liste de propositions) et d'une conclusion (proposition).
- Le moteur d'inférence parcourt les règles et, lorsque toutes les propositions de la prémisses sont présentes dans la base de faits, ajoute la conclusion à la base de faits ; l'algorithme itère jusqu'au point fixe.

Démarche et organisation :

- Définition des TAD Règle et BC en s'appuyant uniquement sur les opérations de la liste abstraite vues en cours (création, accès, test de vacuité, ajout en queue, retrait, appartenance récursive).
- Mise en place d'un moteur d'inférence modulaire utilisant les TAD précédents.
- Interface TUI utilisant *ncurses* pour une interaction plus fluide.
- Environnement outillé : projet C structuré, compilation par *CMake* et exécutable unique.

Note : Le programme a été intégralement réalisé et testé sur linux il est possible que la compilation et exécution soient problématique sur Windows.

Choix de Conception et Structures de Données

Type Abstrait « Liste »

Le projet utilise une implémentation concrète de listes chaînées simples pour représenter les ensembles ordonnés de propositions et de règles. Deux types de listes sont définis :

- **ListProposition** : liste chaînée de propositions
- **ListRegle** : liste chaînée de règles

Structures de données :

- Chaque liste est composée de nœuds (*ListPropositionNode* et *ListRegleNode*) contenant une valeur et un pointeur vers le nœud suivant.
- La liste elle-même maintient des pointeurs vers la tête (*head*), la queue (*tail*) et stocke sa taille (*size*) pour optimiser les opérations.

Opérations disponibles (respectant les principes du cours) :

- **listp_create()** / **listr_create()** : créer une liste vide
- **listp_is_empty()** / **listr_is_empty()** : tester si la liste est vide
- **listp_push_back()** / **listr_push_back()** : ajouter un élément en queue (insertion FIFO)
- **listp_head()** / **listr_head()** : accéder à l'élément en tête
- **listp_remove_first()** : retirer la première occurrence d'un élément
- **listp_contains()** : tester l'appartenance d'un élément (via *listp_contains_recursive()*)
- **listp_free()** / **listr_free()** : libérer la liste et ses éléments

Invariants :

- Si la liste est vide, *head* et *tail* valent *NULL* et *size* vaut 0.
- Si la liste contient un seul élément, *head == tail* et *size* vaut 1.
- Pour toute liste non vide, *tail->next == NULL* (le dernier nœud pointe vers *NULL*).
- La valeur de *size* correspond toujours au nombre réel de nœuds dans la liste.

Type Proposition

Représentation choisie :

Une proposition est représentée par une structure contenant :

- **name** : chaîne de caractères dynamique (allouée sur le tas) identifiant la proposition.
- **negated** : entier (0 ou 1) indiquant si la proposition est niée (–).

Identifiant et égalité :

- L'identifiant d'une proposition est son nom combiné à son état de négation.
- Deux propositions sont égales (*proposition_equals*) si et seulement si elles ont le même nom (comparaison de chaînes via *strcmp*) et le même état de négation.
- Cette représentation permet de distinguer une proposition A de sa négation –A.

Gestion de la mémoire :

- Le nom est copié lors de la création (*proposition_make*) pour garantir l'indépendance des données.
- La libération (*proposition_free*) désalloue explicitement la mémoire du nom pour éviter les fuites.

Type Abstrait Regle

Structure :

Une règle est composée de trois champs :

- **premises** : liste chaînée de propositions formant la prémisse (conditions de la règle).
- **has_conclusion** : drapeau booléen (0 ou 1) indiquant si la règle possède une conclusion définie.
- **conclusion** : proposition déduite lorsque toutes les prémisses sont satisfaites.

Opérations principales :

- **regle_create()** : initialise une règle avec prémisse vide et sans conclusion.
- **regle_add_premise()** : ajoute une proposition à la prémisse en queue.
- **regle_set_conclusion()** : définit la conclusion de la règle.
- **regle_premise_contains_recursive()** : teste récursivement l'appartenance d'une proposition à la prémisse.
- **regle_remove_premise()** : supprime une proposition de la prémisse.
- **regle_premise_is_empty()** : teste si la prémisse est vide.
- **regle_premise_head()** : accède à la première proposition de la prémisse.

Invariants :

- Une règle nouvellement créée a une prémisse vide et **has_conclusion** vaut 0.
- Une règle est applicable si **has_conclusion** == 1 et toutes les propositions de premises sont présentes dans la base de faits.

Type Abstrait BC (Base de Connaissances)

Structure :

La base de connaissances encapsule une liste de règles :

Opérations :

- **bc_create()** : crée une base vide (liste de règles vide).
- **bc_add_regle()** : ajoute une règle en queue de la base.
- **bc_head_regle()** : accède à la première règle de la base.

- **bc_remove_rule_by_label()** : supprime une règle par le nom de sa conclusion (utile pour la simplification).
- **bc_free()** : libère toute la base et ses règles.

Choix de conception :

- L'encapsulation dans une structure *BC* (plutôt que d'utiliser directement *ListRegle*) permet une évolution future (ajout de métadonnées, statistiques, etc.).
- L'ordre d'ajout des règles est préservé (insertion en queue), ce qui est important pour certains comportements déterministes du moteur.

Démarche d'Implémentation

Pourquoi ces structures :

- **Listes chaînées simples** : adaptées aux besoins du projet (ajout en queue fréquent, parcours séquentiel, suppression occasionnelle). Elles offrent un compromis acceptable entre simplicité d'implémentation et performance.
- **Maintien des pointeurs *head* et *tail*** : optimise l'ajout en queue en $O(1)$ au lieu de $O(n)$.
- **Allocation dynamique pour les noms** : flexibilité pour des propositions de longueur variable sans limite prédéfinie.
- **Séparation claire des types** : encapsulation des opérations selon les principes de l'abstraction vue en cours, facilitant la maintenance et les tests.

Complexité attendue :

- Ajout d'une règle ou proposition en queue : **$O(1)$** (grâce au pointeur *tail*).
- Test d'appartenance dans une prémisse (récursif) : **$O(n)$** où n est le nombre de propositions dans la prémisse.
- Parcours complet de la base de connaissances : **$O(m)$** où m est le nombre de règles.
- Moteur d'inférence : **$O(k \times m \times p \times f)$** où k est le nombre d'itérations, m le nombre de règles, p la taille moyenne des prémisses et f la taille de la base de faits.

Compromis :

- **Mémoire vs performance** : les listes chaînées ont un surcoût mémoire (pointeurs) mais évitent les réallocations coûteuses des tableaux dynamiques.
- **Simplicité vs optimisation** : l'implémentation privilégie la clarté et la conformité aux types abstraits du cours plutôt que l'optimisation maximale (pas de tables de hachage, pas d'indexation).

- **Copie par valeur** : les propositions et règles sont copiées lors de l'insertion dans les listes, garantissant l'indépendance des données au prix d'un léger surcoût mémoire.

Algorithmes des Sous-Programmes demandés

Le nom des algorithmes et variables ne c à leurs contreparties dans le code afin d'améliorer la lisibilité.

Type Abstrait Règle

Créer une règle vide

```
Lexique:
  Résultat : r : Regle
  Donnée : -
Algo:
  Fonction regle_create() : Regle
  Debut
    r.premises <- creerListe()
    r.has_conclusion <- 0
    regle_create <- r
  Fin
```

Ajouter une proposition à la prémisse (en queue)

```
Lexique:
  Résultat : -
  Donnée : r : Regle, p : Proposition
Algo:
  Procédure regle_add_premise(r : Regle, p : Proposition)
  Debut
    Si non(estVide(r)) Alors
      ajouterEnQueue(r.premises, p)
    FinSi
  Fin
```

Créer la conclusion d'une règle

```
Lexique:
  Résultat : -
  Donnée : r : Regle, c : Proposition
Algo:
  Procédure regle_set_conclusion(r : Regle, c : Proposition)
  Debut
    Si non(estVide(r)) Alors
      conclusion(r) <- c
      has_conclusion(r) <- 1
    FinSi
  Fin
```

Tester l'appartenance d'une proposition à la prémisse (récursif)

```
Lexique:
  Résultat : b : Booléen
  Donnée : r : Regle, p : Proposition
Algo:
  Fonction regle_premise_contains_recursive(r : Regle, p :
Proposition) : Booléen
  Debut
    Si estvide(r) Alors
      regle_premise_contains_recursive <- 0
    Sinon
      regle_premise_contains_recursive <-
listp_contains_recursive(tete(premises(r)), p)
    FinSi
  Fin
```

Supprimer une proposition donnée de la prémisse

```
Lexique:
  Résultat : succès : Booléen
  Donnée : r : Regle, p : Proposition
Algo:
  Fonction regle_remove_premise(r : Regle, p : Proposition) :
Booléen
  Debut
    Si estvide(r) Alors
      regle_remove_premis <- 0
    Sinon
      regle_remove_premise <- listp_remove_first(premises(r), p)
    FinSi
  Fin
```

Tester si la prémisse est vide

```
Lexique:
  Résultat : b : Booléen
  Donnée : r : Regle
Algo:
  Fonction regle_premise_is_empty(r : Regle) : Booléen
  Debut
    Si estVide(r) Alors
      regle_premise_is_empty <- 1
    Sinon
      regle_premise_is_empty <- estVide(premises(r))
    FinSi
  Fin
```

Accéder à la proposition en tête de la prémisse

```
Lexique:
  Résultat : q : Proposition
  Donnée : r : Regle
Algo:
  Fonction regle_premise_head(r : Regle) : Proposition
  Debut
    Si estVide(r) Alors
      regle_premise_head <-  $\perp$ 
    Sinon Si estVide(premises(r)) Alors
      regle_premise_head <-  $\perp$ 
    Sinon
      regle_premise_head <- tete(premises(r))
  FinSi
  Fin
```

Accéder à la conclusion d'une règle

```
Lexique:
  Résultat : c : Proposition
  Donnée : r : Regle
Algo:
  Fonction regle_get_conclusion(r : Regle) : Proposition
  Debut
    Si estVide(r) Alors
      regle_get_conclusion <- vide
    Sinon Si has_conclusion(r) = 0 Alors
      regle_get_conclusion <- vide
    Sinon
      regle_get_conclusion <- conclusion(r)
  FinSi
  Fin
```

Vérifier si une règle possède une conclusion

```
Lexique:
  Résultat : b : Booleen
  Donnée : r : Regle
Algo:
  Fonction regle_has_conclusion(r : Regle) : Booleen
  Debut
    Si estVide(r) Alors
      regle_has_conclusion <- 0
    Sinon
      regle_has_conclusion <- (has_conclusion(r) = 1)
  FinSi
  Fin
```

Type Abstrait BC (Base de Connaissances)

Créer une base vide

```
Lexique:
  Résultat : bc : BC
  Donnée : -
Algo:
  Fonction bc_create() : BC
  Debut
    bc.regles <- creerListe()
    bc_create <- bc
  Fin
```

Ajouter une règle à une base (en queue)

```
Lexique:
  Résultat : -
  Donnée : bc : BC, r : Regle
Algo:
  Procedure bc_add_regle(bc : BC, r : Regle)
  Debut
    Si non(estVide(bc)) Alors
      ajouterEnQueue(bc.regles, r)
    FinSi
  Fin
```

Accéder à la règle en tête de la base

```
Lexique:
  Résultat : r : Regle | vide
  Donnée : bc : BC
Algo:
  Fonction bc_head_regle(bc : BC) : Regle
  Debut
    Si estVide(bc.regles) Alors
      bc_head_regle <- vide
    Sinon
      bc_head_regle <- tete(bc.regles)
    FinSi
  Fin
```

Supprimer une règle par le nom de sa conclusion

Lexique:

Résultat : succès : Booléen

Donnée : bc : BC, label : Chaîne

Algo:

Fonction bc_remove_rule_by_label(bc : BC, label : Chaîne) :

Booléen

Debut

nœud <- tete(bc.regles)

TantQue non(estVide(nœud)) Faire

 r <- valeur(nœud)

 Si regle_has_conclusion(r) = Vrai Alors

 c <- regle_get_conclusion(r)

 Si nom(c) = label Alors

 retirer(bc.regles, r)

 bc_remove_rule_by_label <- Vrai

 Retour

 FinSi

 FinSi

 nœud <- suivant(nœud)

FinTQ

bc_remove_rule_by_label <- Faux

Fin

Type Abstrait Base De Faits

Créer une base de faits vide

```
Lexique:
  Résultat : bf : BaseFaits
  Donnée : -
Algo:
  Fonction facts_create() : BaseFaits
  Debut
    bf.facts <- creerListe()
    facts_create <- bf
  Fin
```

Ajouter un fait à la base

```
Lexique:
  Résultat : -
  Donnée : bf : BaseFaits, p : Proposition
Algo:
  Procedure facts_add(bf : BaseFaits, p : Proposition)
  Debut
    Si non(estVide(bf)) Alors
      Si contient(bf.facts, p) = Faux Alors
        ajouterEnQueue(bf.facts, p)
      FinSi
    FinSi
  Fin
```

Tester si un fait appartient à la base

```
Lexique:
  Résultat : b : Booleen
  Donnée : bf : BaseFaits, p : Proposition
Algo:
  Fonction facts_contains(bf : BaseFaits, p : Proposition) : Booleen
  Debut
    Si estVide(bf) Alors
      facts_contains <- 0
    Sinon
      facts_contains <- contient(bf.facts, p)
    FinSi
  Fin
```

Vérifier si la prémisse d'une règle est satisfaite

```
Lexique:
  Résultat : b : Booléen
  Donnée : r : Regle, bf : BaseFaits
Algo:
  Fonction premises_satisfied(r : Regle, bf : BaseFaits) : Booléen
  Debut
    nœud <- tete(premises(r))
    TantQue non(estVide(nœud)) Faire
      p <- valeur(nœud)
      Si estNegatif(p) = 0 Alors
        // Prémisse positive : doit être présente dans les
faits
        Si contient(bf.facts, p) = Faux Alors
          premises_satisfied <- Faux
          Retour
        FinSi
      Sinon
        // Prémisse négée : doit être absente des faits
        p_positive <- créer_proposition(nom(p), 0)
        Si contient(bf.facts, p_positive) = Vrai Alors
          premises_satisfied <- Faux
          liberer_proposition(p_positive)
          Retour
        FinSi
        liberer_proposition(p_positive)
      FinSi
      nœud <- suivant(nœud)
    FinTQ
    premises_satisfied <- Vrai
  Fin
```


Moteur d'Inférence par Chaînage Avant

```
Lexique:
  Résultat : -
  Donnée : bc : BC, bf : BaseFaits
  Variables : changement : Booleen, r : Regle, c : Proposition
Algo:
  Procedure inference_forward_chain(bc : BC, bf : BaseFaits)
  Debut
    changement <- Vrai
    TantQue changement Faire
      changement <- Faux
      nœud <- tete(bc.regles)
      TantQue non(estVide(nœud)) Faire
        r <- valeur(nœud)
        // Tester si la règle peut s'appliquer
        Si regle_has_conclusion(r) = Vrai ET
premises_satisfied(r, bf) = Vrai Alors
          c <- regle_get_conclusion(r)
          Si facts_contains(bf, c) = Faux Alors
            facts_add(bf, c)
            changement <- Vrai
          FinSi
        FinSi
      nœud <- suivant(nœud)
    FinTQ
  FinTQ
Fin
```

Notes

- Le moteur d'inférence applique les règles tant qu'il enrichit la base de faits;
terminaison au point fixe.
- Chaque règle est testée à chaque itération; dès qu'une conclusion est ajoutée, une nouvelle itération commence.
- Les propositions inversées dans les prémisses sont traitées comme l'absence du fait positif correspondant.
- Aucun doublon n'est ajouté à la base de faits (vérification par *facts_contains* avant ajout).

Tests Unitaires

Stratégie de Test

Pour valider l'implémentation de chaque type abstrait et du moteur d'inférence, une suite de tests unitaires autonomes a été développée. Chaque test vérifie :

- **Création et initialisation** : les structures sont correctement allouées et initialisées.
- **Opérations élémentaires** : ajout, accès, suppression, tests de prédicats (appartenance, vacuité).
- **Invariants** : la structure respecte ses contraintes (ex. : cohérence de size avec le nombre réel de nœuds).
- **Cas limites** : structures vides, opérations sur NULL, mémoire correctement libérée sans fuites.

Suite de Tests Implémentée

Les tests sont organisés par type abstrait et localisés dans le répertoire tests/ :

test_proposition.c :

- Création de propositions simples et négatives.
- Comparaison d'égalité (même nom et même état de négation).
- Libération mémoire.

test_list_proposition.c :

- Création d'une liste vide.
- Insertion en queue (opération FIFO).
- Test d'appartenance (récursif et itératif).
- Suppression du premier élément et suppression totale par nom.
- Cas limite : listes vides, éléments non présents.

test_regle.c :

- Création d'une règle vide (prémisse vide, pas de conclusion).
- Ajout de propositions à la prémisse.
- Définition et récupération de conclusion.
- Test d'appartenance récursive dans la prémisse.
- Suppression de propositions de la prémisse.
- Accesseur de tête.

test_bc.c :

- Création d'une base de connaissances vide.
- Ajout de plusieurs règles en queue.
- Accès à la première règle.
- Suppression de règles par label de conclusion.
- Cas limite : tentative de suppression répétée.

test_inference.c :

- Construction d'une base de connaissances complexe.
- Exécution du moteur d'inférence sur des faits initiaux divers.
- Vérification des faits déduits attendus.
- Test de chaînage multi-étapes (règles en cascade).
- Validation du point fixe (arrêt de la propagation).

Résultats de la Validation

Tous les tests passent avec succès :

```
test_proposition: OK
test_list_proposition: OK
test_regle: OK
test_bc: OK
test_inference: OK
```

Exécution (exemple) :

```
cd build
./test_proposition
./test_list_proposition
./test_regle
./test_bc
./test_inference
```

Interface Utilisateur Interactive

Motivation et Objectifs

Au-delà du moteur d'inférence fonctionnel, une interface utilisateur graphique (en mode texte) a été développée pour faciliter l'exploration et la manipulation de bases de connaissances. Les objectifs principaux sont :

1. **Visualisation intuitive** : afficher graphiquement les règles sous forme de graphe ASCII avec mise en évidence des faits vrais.
2. **Interaction en temps réel** : permettre de modifier les faits de base (entrées) et observer immédiatement les conséquences inférées.
3. **Construction dynamique** : offrir la possibilité de créer et éditer des bases de connaissances de manière interactive.

Choix Technologiques

Bibliothèque ncurses :

- Utilisation de la bibliothèque **ncurses** pour créer une interface en mode terminal avancé.
- Avantages : portabilité (Linux, macOS, Windows avec support approprié), légèreté, pas de dépendances graphiques lourdes.
- Support UTF-8 pour les caractères de tracé (`|` , `├` , `└` , `┌` , `┐`) améliorant la lisibilité du graphe.

Fallback en mode texte simple :

- Un mode `--text-only` (-t) est proposé pour les systèmes sans ncurses, affichant simplement les faits avant/après inférence et un graphe ASCII statique.
- Garantit l'accessibilité universelle du programme.

Conclusion

Ce projet a permis de concevoir et d'implémenter un système expert complet par chaînage avant en langage C, en respectant les principes d'abstraction des types de données vus en cours. Les structures de données choisies (listes chaînées simples pour les prémisses, règles et faits) offrent un bon compromis entre simplicité d'implémentation et performance pour des bases de connaissances de taille modérée.

Le moteur d'inférence développé garantit la terminaison par convergence vers un point fixe et produit des résultats cohérents sur l'ensemble des jeux d'essais. L'approche modulaire adoptée (séparation claire entre TAD Regle, BC, BaseFaits et moteur d'inférence) facilite la maintenance et l'évolution du code.

L'ajout d'une interface utilisateur interactive basée sur ncurses constitue une valeur ajoutée significative, transformant un outil de démonstration académique en un environnement d'exploration pédagogique permettant de visualiser en temps réel le comportement du système et de construire dynamiquement des bases de connaissances.

Les perspectives d'amélioration incluent notamment l'ajout d'un système de persistance (sauvegarde/chargement de bases), un mode de traçage détaillé du processus d'inférence, et l'optimisation des structures de données pour des bases de grande taille (tables de hachage, indexation). Le projet démontre néanmoins une maîtrise satisfaisante des concepts fondamentaux des systèmes experts et de la programmation modulaire en C.